

1. introduction

本文属于自己的技术自嗨。实在是每次查问题，东查一下，西查一下真的烦到自己是了。所以问了 AI 总结，以后用得上再说，用不上拉倒，毕竟我也才是一个本科毕业生。

我希望总结出自己的调试方法，而不是每次一遇到问题都会慌。

个人的总结：任何的 bug，本质上都是只有一种：**系统的客观物理/逻辑运行轨迹，偏离了你大脑中预期的“心智模型 (Mental Model)”**。

基于这个总结，自己总结出一个属于自己的调试认知，一个系统演绎调试法。

2. 调试的心智认知

1. 剥离事实——建立事实清单

无论这一阶段问题表露出何种现象/具有何种表现，都只认识事实，因为表现是“**经过了多层的封装后暴露出的假象**”，充满误导。

这一步做：

- **核心动作**：切断一切主观推断，只记录机器输出的客观事实（不要去辩驳，输出是什么就是是什么）。
- **准则**：这一步，不问“为什么”，只问“是什么”。把所有的 log、dump、波形当作案发现场的痕迹，它们只是事实，不是结论。
- **思维训练**：
 - **错误视角**：“系统死机了。”（这是主观判断，因为可能只是显示输出挂了，后台还在跑）。
 - **正确视角**：“PC 寄存器指向了未分配的内存地址 `0x00000000`。”或者“模块 A 在收到事件 B 后，没有在 5ms 内发出事件 C。”

2. 契约重构——找回丢失的参考系

遇到 bug 时，之所以觉得系统出问题了，本质上是因为系统存在一个“正常的参考系”（脑海里默认存在应该正常执行是这样的）。

而很多时候排查问题毫无章法，是因为脑子里没有这张“正确的运行flow图”。

可以总结：任何系统都是由一层层的**契约（接口、协议、时序）**组成的。

这一步做：

- **核心动作**：在大脑中（或纸上）画出数据流/控制流的拓扑图，并标明每个节点的“输入契约”和“输出契约”。
- **准则**：无法被建模的系统是无法被调试的，如果对正常数据流走向一无所知，那根本就不可能知道数据在哪里消失了。
- **思维训练**：
 - 不要去读每一行代码，去理清**模块与模块之间的边界**。
 - 内部发问：要让模块 B 吐出期望的结果，模块 A 必须以什么样的时序、格式、状态给它喂数据？

建立pipeline学习，那具体这个粒度要细致到什么程度？

3. 正交隔离——找到“契约破坏点”

在一个庞大的系统中，直接找错误是不现实的，所以需要使用经典的**二分法**不断地对切系统边界，直到缩小到最小不可分割单元。

这一步做：

- **核心动作**：设计极端的边界条件，把问题空间一刀切成两半，证明问题在左边还是右边。
- **准则**：永远不要试图一次性从头排查到尾。在整个拓扑结构的中点“切一刀”，观察输出，排除掉 50% 的可能性。
- **思维训练（切分维度的艺术）**：
 - **软硬隔离**：是代码发错指令了，还是硬件收到指令没动作？（在交界处打桩、抓波形）。
 - **时空隔离**：错误是每次必现（空间逻辑错误），还是特定时间、特定并发下才现（时间/时序错误）？
 - **变量隔离（控制变量法）**：如果系统有 A、B、C 三个特性同时在跑，关掉 B 和 C，A 还会报错吗？

4. 假设与证伪 (Hypothesis & Falsification) ——使用合理的调试工具/探针

当通过正交隔离把包围圈缩小到某一个特定的模块或者机制的时候，先不要着急去改代码/硬件试错，要做的是：提出假设，并设计实验去推翻它！

这一步做：

- **核心动作：**提出 2-3 个能导致当前事实的假设，然后设计一个“只有当该假设成立时，才会出现特定结果”的观测手段。
- **准则：**不是用代码修复 Bug，而是用数据证明 Bug！！你的探针（工具）只是为了获取实证数据！
- **思维训练：**
 - 假设：“我怀疑是多线程抢占导致了共享变量被污染。”
 - 探针设计：“如果我的假设成立，那么在访问这个变量前加上自旋锁/关中断，问题就会消失。”或者“如果我的假设成立，那么抓取 CPU 的调度历史（trace），一定能在出错的微秒级瞬间看到任务切换。”

面对任何 Bug（无论你是搞上层应用、底层驱动、还是架构设计）→ **不要猜** → 提取机器层面的绝对事实 → 画出该功能涉及的组件拓扑与契约 → 用二分法切断链路寻找哪个组件违约 → 针对违约嫌疑人提出导致违约的假设 → 用工具获取数据证明假设。

3. 诊断记录模板

来源于立芯嵌入式：<https://twd6onxsxva.feishu.cn/docx/RjnOdcZ3FoxcQlxEJYpceaTjnxg>

一、问题的描述

1. 问题的表现是什么样的？

关于问题的表现的描述，素材应尽量详细，多附上一些截图/录屏/复现文档

2. 问题的复现路径

比如：

- i. 工程代码附上
- ii. 说明复现细节：使用串口输出还是RTT，波特率

3. 正常的预期是什么样的

正常表现应该是什么样的？比如串口应该输出“Hello World!”

二、问题产生的可能原因分析

1. 初步 CheckList 确认

比如某个跑 FreeRTOS 的例子：

- i. 排除硬件问题：找一个确定可以运行的软件，先跑一下，确认硬件没有问题
- ii. 程序可能爆栈了：在启动文件里调整下栈大小，如果是在RTOS里，则调整一下任务的栈大小
- iii. 程序可能被过度优化：调整优化等级，建议0
- iv. 程序可能进入死循环：进入调试模式，然后全速运行程序然后按暂停，看是否进入 **Hardfault**，然后用栈回溯)
- v. 可能程序执行错误：打印每个相关函数的返回值！
- vi. 可能指针为空：打断点到指针运行处，看指针里面的值是不是 0x00000000
- vii. 可能 API 接口用错：Freertos 最好使用 Freertos 本身的 api，比如使用 queue.h 文件里面的函数，task.h 里面的函数，而不是 CMSIS os_V2.h 里面的函数。
- viii. 可能有些程序片段根本没执行到：比如按键操作如果用 polling 的方式，

它运行很快，打断点不方便观察，应该在每个状态机或 if 语句里放置 `printf("1/r/n"); printf("2/r/n");` 等

- ix. 可能有些线程被饿死了：在线程里面加一些 `VTaskDelay(100)` 延时，防止有些线程被饿死。
- x. 可能有些线程没有 `while(1)` 循环：检查是不是有些线程没有死循环，直接退出了
- xi. 如果有互斥量和信号量的使用，项目卡死，则先尝试关闭它们，看会不会卡死（一般情况下是死锁），再尝试逐个成对打开。
- xii. 所有的局部变量，全局变量一定要赋初值

2. 提出可能的原因

比如可能是任务调度导致的串口打印不完全，关掉任务调度试一试

三、设计实验，验证可能的原因和猜想

详细描述修复问题的技术方案，相关示意图、流程图、时序图

四、验证实验

第一次实验

1. 实验序号

- 注明这是第几次实验（每次都需修改）
- 实验时间（每次都需要更改）

2. 实验环境

- 本次测试的测试环境
芯片型号、固件版本、电源供电的内容、有其他实验环境也需要标注
- 本次实验的相关文档

比如 SoC/MCU 的 Datasheet, User manual等

3. 实验步骤

- i. 记录下如何启动的 MCU/SoC, 是通过电源上下电还是按键复位
- ii. 其他与本次实验相关的步骤

4. 实验结果

- 输出本次的实验结果
分析一下本次实验有什么新的发现
- 实验分析
 - i. 对比本次实验和上次实验步骤的区别
 - ii. 对比本次实验和上次实验结果的区别
 - iii. 是否解决问题? 若无, 设计下一次实验

4. 个人进一步看法

尽管上面的这个认知模型+排查说的这么抽象和宏大, 但实际上随着个人水平的提升和时间效率的约束, 遇到问题时并不是每次都需要使用如此复杂完整的内容, 浪费时间, 故应该合理地使用其中的内容。

所以这里针对两个自己想到的点去考虑。

错误分级:

完全没有必要每一次都使用这么复杂的记录模板+心智认知模型。我们决大多数时候遇到的问题都是比较简单的! 所以对于我自己来说, 我这么分级:

- L1: 常识性/语法性错误 (AI 直接解决区)
 - **特征:** 明确的编译报错、常见的内核 Panic 签名 (如明确指出某行代码 NULL pointer dereference)、或是某个 API 返回了标准错误码。
 - **做法:** 根本不需要上面的什么步骤, 直接拿着报错信息搜网络、问

LLM+官方文档。快速扫一眼问题是什么即可。

- L2: 确定性逻辑错误 (轻量级调试区+配合 AI)
 - **特征:** 问题 100% 必现。比如配置了 DMA 但是没出数据, 某项功能点进去就闪退。
 - **对策:** 问题因为配置漏了或者时序错了一步。画一个简单的流程, 打几个 `print` 或者挂上 JTAG 单步走一下, 看代码走到了哪个分支。
- L3: 非确定性/系统架构级问题 (深度演绎调试区 + 配合AI)
 - **特征:** 偶现的卡顿 (Glitch)、系统概率性死锁、内存泄漏、休眠唤醒 (Suspend/Resume) 失败、跨核通信 (如 Mailbox) 时序错乱。
 - **对策:** 进一步使用个人总结的“系统演绎调试法”和重型模板。不要盲目搜网络或乱调代码。

运行 flow 图

在相对比较大的系统中, 是永远不可能在调试开始之前就掌握模块的所有底层信息细节的! 你什么都知道了, 那还有什么bug?

就比如, 非要把 Linux 音频子系统从 `ALSA Core` 到 `ASoC` 再到底层的 `DMA` 驱动全看懂才开始查音频卡顿, 项目早就延期了!!

自己认为更加合理的做法是: “**Just-in-Time (即时) 契约构建**”与“**宏观二分法**”交替进行。

1. 宏观隔离 (先切分黑盒)

我根本不需要知道盒子里面是什么, 只需要知道边界!

比如音频不出声, 第一刀可以选择直接切在 User Space 和 Kernel Space 之间。用 `aplay` 往声卡节点灌一个标准的 `wav` 文件。(总之是你合理的思考推断)

- 如果能出声，说明 Kernel 底层和硬件全都是好的，问题在 App 或者 Framework 层。
- 如果不出声，说明问题在 Kernel 层或硬件。

2. 微观重构（只学被包围的那一部分）

当确定了问题在 Kernel 层，并且进一步二分发现 DMA 确实没有收到数据。

此时，才需要去查阅资料，构建“这一个微小环节”的心智模型。

直接去翻看 Datasheet 里的 DMA 章节，或者阅读 Linux 内核中该 DMA 控制器的驱动代码，明确它需要怎样的“输入契约”，哪里的参数、状态不对！

补充一点画图的：

脑海中的数据流向图是一个**由粗到细、动态迭代**的过程。开始排查前，画出粗粒度的模块图；在二分法切分到可疑区域后，再针对这个小区域画出极其精确的时序/状态机图。

个人总结调试用（一）

这部分应该最常用的，以简单为主，实际上这个才是我很早之前做的笔记的版本（但是我的语言描述很匮乏，只能用例子），这里以这个 `uboot shell` 中 eMMC 启动报错、自己之前遇到的是不是 SD 卡挂载不成功的为例子。

```
1 => mmc list
2 mmc@fe320000: 1
3 mmc@fe330000: 0
4
5 => mmc dev 0
6 unable to select a mode: -5
```

来源：[eMMC 问题定位](#)（这里我是还没看这个文章的，相等之后具体查 bug 看看自己的思考和资深工程师的思考差异）

1. 语义映射——报错在代码中究竟意味着什么？

不能停留在“模式选择失败”的表面翻译，必须将日志映射回源代码的物理执行点。

- **动作：** 全局搜索 U-Boot/Kernel 源码。找到报错的函数
- **追问本质：** 返回值 `-5` 在 Linux/U-Boot 错误码体系中通常代表 `-EIO` (I/O Error)。这说明什么？Gemini 的回答：

说明这**不是**软件逻辑判断出错了（比如参数传错通常是 `-EINVAL`），而是 Host 控制器在通过 CMD 线向 eMMC 发送命令请求切换总线宽度/速度模式（如 HS200/HS400）时，底层的硬件响应超时或 CRC 校验失败了。

2. 状态机对齐——系统挂在哪个阶段了

- **动作：** 脑海中或纸面上必须有该模块的协议流转图。以 eMMC 为例：上电 -> 发送 CMD0 复位 -> CMD8 查电压 -> CMD1 查容量 -> CMD3 获取 RCA -> CMD6 切换模式 (这里挂了)。
- **追问本质：** 既然 `mmc list` 能看到节点，说明前面的基本识别 (CMD0~CMD3) 都过了，底层时钟、供电、基础通信是好的。死在 CMD6 切换高速模式，本质立刻收敛：要么是 VCCQ 的 1.8V 供电没切过去，要么是高速模式下的 IO 驱动能力 (Drive Strength) 或采样时序 (Tuning) 需要校准。

3. 横向责任划分——划分责任归谁管的

- **动作：** 通过极端测试切断链路。
- **追问本质：** 比如 STM32 SD 卡偶发挂载失败。既然 SD 卡在电脑上好的 (Device 没问题)，那么边界就缩小到了 STM32 的 SDMMC/SDIO 外设控制器相关驱动代码、PCB 走线 (物理总线)、`FatFS` 与底层驱动的接口代码 这三块。

4. 动态变量降维——控制变量/二分法不断地找问题

对于“偶尔成功，偶尔失败”的问题，静态代码审查通常无效，因为它的本质是系统运行时的动态干扰。

- **追问本质：** 偶发失败的本质只有三类：
 - i. **时序抢占 (软件并发)：** 比如 STM32 在搬运 SD 卡数据的 DMA 中断，是不是被一个更高优先级的实时任务 (或者其他硬件外设中断) 抢占了，导致 FIFO 溢出？

- ii. **信号完整性（硬件电气特性）**：高速读写瞬间，电流飙升，是不是板子上的 LDO 稳压器扛不住，导致瞬间电压跌落（Brownout）引发 SD 卡内部复位？是不是 CMD/DAT 线路上缺少合适的上拉电阻，导致高频信号变成了正弦波甚至毛刺？
- iii. **内存踩踏（指针问题）**：其他毫不相干的任务越界，污染了 `FatFS` 的缓存区。

个人总结调试用（二）

Gemini 基于我上面的理解生成的：

1. 问题分级与事实封存 (Triage & Fact Freeze)

- 判断级别：这是 L1 (常识), L2 (必现逻辑), 还是 L3 (偶发/时序/并发)? (仅 L3 往下走)。
- 记录绝对事实：
 - 发生时间与频率（如：压测 10 小时出现 1 次）。
 - 最后一句有效的 Log 是什么？（不要加主观翻译）。
 - 现场留存了什么？（Dump 文件、Oops 信息、寄存器现场）。

2. 宏观边界隔离 (Macro Boundary Isolation) (动作：先切黑盒)

- 寻找中点：这个功能的数据流，最明显的物理或软件边界在哪里？（例：应用层 vs 驱动层；主核 CPU vs 协处理器/Sensor Hub；SoC 控制器 vs 外部芯片）。
- 实施切割：设计一个极简测试用例（如使用内核自带的 dummy 驱动，或直接操作 `/dev/mem` 读写寄存器）来切断链路。
- 结论：确认问题落在左半边还是右半边。

3. 局部契约重构 (Micro Contract Modeling) (动作：针对被锁定的半边，精准突击学习)

- 聚焦靶点：既然锁定在模块 A 和 B 的交界处，它们之间的数据交换协议/

时序是什么？（这里才是需要翻 Datasheet、读核心源码、了解 ELF section 分布机制的时候）。

- 绘制微观图：画出这一个小区域的正确流转图（比如：发起中断 -> 清除标志位 -> 触发 DMA -> 释放 Spinlock）。

4. 假设驱动与探针部署 (Hypothesis & Probing)

- 提出假设：基于微观图，列出 2-3 个可能破坏契约的嫌疑点。（例：是不是中断抢占导致标志位没清？是不是 Cache 一致性没同步导致 DMA 读了旧数据？）
- 部署探针：
 - 我不改逻辑，我只加观测。
 - 使用 `ftrace` 抓调度延迟，使用 `perf` 看函数开销，或者配置硬件触发逻辑分析仪抓波形。
- 收集数据：探针抓回来的数据，是否印证了假设？如果不印证，推翻重来。

5. Phase 5: 修复与防劣化分析 (Fix & Regression Analysis)

- 执行修复：改代码验证。
- 灵魂拷问（极为重要）：这个 Bug 为什么在最初的架构设计或 Code Review 中被遗漏了？有没有类似的同类代码也存在这种风险？

5. 融入 AI 调试

TODO, RAG, 进一步迭代自己的使用的，但自己很懒。